

HighGround: full project audit

One file with everything the team needs to pitch, film, submit and keep building. It covers what we built, how it works, the tech stack we use and the stack still planned, every number we may quote (with its caveat), what is done and what is not, how to run it, the pitch, the video, and the next steps.

Written **Sun 11 Oct 2026, 09:45 IST**. Numbers are copied from the model output files (data/out/web/proof.json, runs.json, hospitals.json, parking.json, streets/index.json, graph/meta.json, offline/manifest.json) and from test logs. **If you quote a number, quote it as written here, with its caveat.**

Companion files:

File	What it is
docs/OFFLINE_AND_NAVIGATION_PLAN.md	Offline and navigation plan with sourced Chennai facts
docs/LANDING_PAGE_CONTEXT.md (+ PDF)	Brief for the landing page
docs/VIDEO_CUT.md	Video cut list with voice-over lines
docs/DEMO.md	Verified demo streets
docs/blog_draft.md	Blog draft
PROGRESS.md	Full engineering log
CLAUDE.md	Original spec

Repo: https://github.com/Divue/high_ground

0. Status at a glance

Item	State
Deadline	Sun 11 Oct 2026, 20:00 IST (online submission; confirm on the wemake devs.org/aws/env form)
Flood model (16 storms) + validation + ANUGA cross-check	Done. v3 shipped. v4 was tested and rejected by a rule written before the run
Web app (Tonight, replays, time-lapse, What if, Hospitals, Proof, About, assistant UI)	Done , working on the production build (<code>vite preview</code> :4173)
Offline mode (Save for offline, battery saver map, help numbers, My flood plan)	Done and reviewed (Sun 00:55–09:00). Offline gate passes with only the saved pack on the phone
Flood-safe navigation (on foot / two-wheeler / car, directions, live GPS, Preview the drive)	Done and reviewed. 0 rule violations on 36 test trips
AWS backend (SAM: S3, CloudFront, 4 Lambdas, API Gateway, DynamoDB, SNS, EventBridge Scheduler, Bedrock, Amazon Location, Amplify)	Code done , tested against mocked AWS (moto). Not deployed ; the user setup is missing (section 16)

Item	State
Live URL, real alert email, live assistant	After deploy. These are the two slots in the rough cut
Demo video	6 shots recorded at 1080p/30 fps and checked frame by frame. Rough cut 2:30 at <code>review/p5/footage/rough_cut.mp4</code> (720p copy at <code>docs/video/HighGround_rough_cut_720p.mp4</code>). Cut list in <code>docs/VIDEO_CUT.md</code>
Demo streets	<code>docs/DEMO.md</code> lists only streets verified by typing them into the search box
Landing page	Teammate. Context PDF updated with the positioning change and the new features
Blog	<code>docs/blog_draft.md</code> (numbers corrected); needs the offline/navigation paragraph
Reviews	Design critic ×3, QA ×5, model reviewer ×2, judge ×2. Every blocker and major fixed (section 9)
Phase gates	P1 passed (tag <code>p1</code>). P2 : mass balance, depth maps, Pallikaranai wet and T. Nagar mostly dry pass; "Velachery wetter than T. Nagar" fails (disclosed). P3 passed (ranking test). P4/P5 need the deploy

Latest scores (judge #2, Sun 08:30, before the search fix and rough cut):

Criterion	Score
Idea and impact	8
Built on AWS	4 (7–8 once deployed and filmed)
Design and usability	7
Execution	6
Demo readiness	4

Its top suggestions were to fix the street search, verify the demo streets and lock the footage; all three are done. What remains is **deploy and film the email and the assistant**.

1. The problem and the pitch

One line: HighGround shows Chennai residents where floodwater will go tonight, street by street, and what to do about it. It keeps working when the power and the network go down.

The questions it answers:

1. Will my street flood tonight? How deep, and when?
2. Where can I park my car on dry ground, and by when must I move it?
3. Which way can I go, on foot, by two-wheeler or by car, without crossing water?
4. Which hospitals get cut off?
5. Email me if this changes.
6. And if the power and towers go down: who do I call, how do I send my location, and how do I get to dry ground offline?

Why it matters (sourced, safe to say):

- **Flyover parking.** Every monsoon Chennai parks its cars on flyovers. In Cyclone Fengal (2024) the Velachery, Pallikaranai, Medavakkam and Mint flyovers filled with cars ([Deccan Herald](#)).
- **Insurance.** The 2015 floods produced Rs 4,800 crore of insurance claims, mostly motor ([Business Standard](#)).
- **Hospitals.** In 2015, 18 ICU patients at MIOT died when floodwater reached the generator room ([CS Monitor/AP](#)).
- **Power cut on purpose.** On 4 Dec 2023 (Cyclone Michaung), 712 of Chennai's 1,814 11 kV feeders were switched off ([TNM](#)).
- **Phone networks fall over.** On 5 Dec 2023, 30% of the city's 42,747 mobile towers were down ([The Week/PTI](#)). In 2015, networks were mostly down for about 3 days ([TNM](#)).
- **Two-wheelers dominate.** Chennai had 54 lakh two-wheelers and 11.75 lakh four-wheelers in 2019 ([Deccan Chronicle](#)), so two-wheeler is the default navigation mode.

Positioning (from the spec; still valid): Chennai already has CFLOWS, a government flood warning system built for officials. HighGround is for the residents who still park their cars on the Velachery flyover every storm because nobody tells them where the water will go. We do not claim a better model than CFLOWS; we claim a tested one, built for citizens.

Positioning update (important): two official tools also reach the public.

- **Chennai Flood Monitor** (RTFF & SDSS, the state, reported operational since Oct 2025) claims street-level inundation forecasts for vulnerable areas.
- **TN-Alert** (state app, 5 lakh+ installs) sends official alerts.

So **never say we are the only street-level forecast**. Say what we add:

Official systems tell you a flood is coming. HighGround tells you what to do on your street: where to move your car and by when, which way to go without crossing water, which hospitals you can still reach, and it keeps working when the power and the network go down. And we show, honestly, how well the model did on 2015.

Taglines:

- "Where the water goes tonight, street by street."
- "Know before the water does."
- "Every monsoon, Chennai parks its cars on flyovers. Now you know where, and by when."
- "Saved before the storm. Works without internet."

2. The event and how we score

Event: Environmental Hacks, event 02 of the Bharat Builds Tour (WeMakeDevs × AWS), Heat and Water track. Hybrid, Oct 8–11, 2026.

Rules:

- teams of 1–4;
- built during the event;
- must use AWS **and show it in the video**;
- a 3-minute video;
- a blog on AWS Builder Center for the blog prize.

Criterion	Our evidence
Idea and impact	One decision per street ("Move your car to X by 2 AM"). Routes around water by mode. Hospitals cut off. Offline help built on sourced outage facts. Two-wheeler first
Built on AWS	12+ AWS services each doing real work (section 6); open data read straight from the AWS Open Data registry; one SAM template. Must be shown live in the video
Design and usability	Full-screen 3D night city. Plain words ("Waist-deep", "too deep for scooters from 9 PM"). Amber only for safety, red only for 112. Phone layout and live-directions view
Execution	A real 2D flood model (16 storms, exact mass balance), an independent second model, honest held-out validation, a routing engine in the browser, offline PWA, 5 QA rounds
Demo video	Frame-perfect footage and a 2:30 rough cut with title cards; two slots for the live AWS proof

3. The product (what a resident sees)

One full-screen 3D night map of Chennai with floodwater on it.

Screen / feature	What it does
Opening	The camera starts low over the Bay of Bengal and flies to a Velachery street in about 5 s. A title line reads "Chennai, 6 PM. Cyclone Michaung, 2023, replayed as if it were tonight." Rain starts, the water rises, and the answer card appears
Tonight (hero)	Type a street and the camera travels across the city to it; a pulse marks the street; the night replays from 6 PM and the water rises hour by hour. The answer card shows: • depth in body words ("Waist-deep") with cm and a "from the model" chip • when the street becomes too deep for scooters • a picture of an adult, a scooter and a car against the water line • the decision: "Move your car to Velachery MRTS Bridge (flyover, 450 m) by Tue 2 AM" with Take me there • Email me if this changes; My flood plan · Save for offline; a sticky footer: <i>In danger, call 112 · Help numbers · Battery saver</i> • search a street name and it answers for that street's deepest stretch near the point, saying how many stretches and their range
Replay a storm	Real storms replayed as if tonight: Dec 2015 (with the reservoir release), Cyclone Michaung 2023, Cyclone Fengal 2024
Watch the whole storm	A ~30 s time-lapse on the storm's real dates, with captions from the model files. Pause, resume, watch again, back to my street
What if	Rainfall slider over the computed storms (50–400 mm), mean or high tide. Shows "% of the city too deep for scooters" and "hospitals cars cannot reach"
Hospitals	46 major hospitals, which are cut off at each storm size, and a route in from the main roads that avoids flooded streets
Proof	"Did the model get 2015 right?" A swipe split-screen of the model vs the streets residents reported, the honest result on one shared scale, the ANUGA cross-check, and "For experts" sections
About the model	Limits in plain words, data credits, animations on/off
Ask HighGround	Strands Agents on Bedrock; answers only from model tools; every number checked and marked "from the model"; 112 first in danger. Shows "offline" until deployed
Take me to dry ground (new)	By: on foot / two-wheeler (default) / car. To: the card's parking, dry parking, a hospital, high ground or a saved place. • Avoids water 10 cm (foot, two-wheeler) or 20 cm (car) at the hour you'd reach each street, looking one hour ahead.

Screen / feature	What it does
	• Closes subways in rain; keeps to one-way streets. • Plans for the hour on the timeline (drag to replan); shows the dashed usual way it avoids, step-by-step directions, and the honesty line above Start. • If there is no route, it says so, with when a way out opens again and "stay put / 112"
Live directions (new)	Start follows your GPS (no mobile data needed) with one big instruction, a turn arrow and the distance. It speaks prompts ("Mute voice"), keeps the screen on, and re-plans after 3 off-route fixes. On a phone, only the map and the instruction show. Preview the drive plays the route, for demos
Save for offline (new)	One tap stores the app, the map, your street (+ up to 2 more places, 3 km around each, every storm), the road network with depths, terrain, and the forecast. Offline, the app opens from the phone and shows "Offline · forecast from 5:24 PM (5 h ago)", and the map switches to battery saver: flat, streets coloured by depth
Help numbers (new)	Full-width tap-to-call rows: 112 (red), 1913 GCC, 1070 state EOC, 1077 district EOC, 108, 101, Minnagam 94987 94987 (fallen wires). Share my location by SMS / WhatsApp / share / copy (GPS, falling back to the chosen street)
My flood plan (new)	An image and a plain-text plan: street, worst depth and time, the decision, dry parking, a schematic route, the nearest hospital still reachable by car, numbers. Replays carry a band " Practice plan: a replay of ... Not a forecast. "; every plan is dated
#admin (hidden)	"Run and send alerts" for a chosen rainfall, to trigger a real alert email for the video

Design language:

- **Concept:** Chennai at 2 AM mid-storm; the water is the brightest thing on screen.
- **Colours:**
- background #050D12, land #0A161C, text #D6E0E4;
- shallow water #86E6EC, deep water #1E7BC6;
- **amber #F2A541 only for safety** (parking, the suggested route, destination);
- **red #E5484D only for 112.**
- **Type:** Anek Latin (the readout widens with depth) and Hind, self-hosted.
- **Copy:** sentence case, plain words, no "safe route" wording.

4. How it works

4.1 Architecture

OFFLINE (Python on a laptop) Copernicus 30 m elevation (AWS Open Data) ESA WorldCover land cover (AWS Open Data) OpenStreetMap (buildings, roads, drains, hospitals, flyovers, one-ways, subways) Rain scenarios + 3 real storms → terrain conditioning → 2D flood model (16 storms, numba) → ANUGA cross-check (Velachery) → validation vs 2015 reports → street / route / hospital / texture / navigation flags / offline manifest → S3 + CloudFront (versioned pack)	ONLINE (AWS) EventBridge Scheduler (every 3 h) → Lambda forecast-check: Open-Meteo rain for 4 points → pick the 2 nearest storms → current.json on S3 → risk changed? Bedrock writes 2 plain sentences → SNS email API Gateway → Lambda subscribe (DynamoDB + SNS filter policy) API Gateway → Lambda assistant (Strands Agents on Bedrock) API Gateway → Lambda geocode (Amazon Location, Nominatim fallback) Amplify Hosting: the PWA (sw.js no-cache)	BROWSER / PHONE (PWA) MapLibre GL 3D city + terrain three.js water: hourly depth textures blended on the GPU Navigation worker: time-dependent search over 206k road edges, directions, live GPS Service worker: app shell precached, saved packs served first, forecast network-first Battery saver: flat map, streets coloured by depth All numbers from model files
---	--	--

Diagram image: docs/architecture.png. It predates offline/navigation; add a "service worker + offline pack" box if regenerating.

4.2 The flood model in plain words

1. **Ground.**
2. The Copernicus GLO-30 elevation model (30 m) includes rooftops and trees.
3. We removed **371,679** OpenStreetMap buildings, tree canopy (ESA WorldCover), bridges and flyovers, and other tall objects, then filled and filtered the gaps.
4. We carved waterways by type (rivers 2 m, canals 1.5 m, streams 0.6 m, drains 0.4 m) and capped fake closed pits at 1 m.
5. Dense buildings go back as +3 m walls that water flows around.
6. Grid: $836 \times 1440 \approx 1.2$ million cells, EPSG:32644.
7. **Water.**
8. A 2D local-inertial shallow-water solver (Bates et al. 2010) in numba: CFL 0.5, a Froude cap, and a donor-cell outflow limiter. **Mass error 0.000% on every run.**
9. Rain is added to every cell. Drains are one uniform capacity (10 mm/h, a stated assumption). Water soaks into pervious land.
10. The sea is held at mean or high tide. The edges of the box let water out, except where the Adyar enters.
11. About 15 min per storm on a 2-core laptop.
12. **Storms (16 runs).**
13. 12 design storms: 50–400 mm in 24 h, front-loaded, each at mean and high tide.
14. 1–2 Dec 2015, 374 mm, run with and without the **Chembarambakkam release** (CAG/PWD audit timeline, peak 29,000 cusecs held 21 h; a documented assumption).
15. Cyclone Michaung 2023, 415 mm; Cyclone Fengal 2024, 114 mm.
16. Storm timing from ERA5, scaled to IMD totals.
17. **Per street:** 168,156 street pieces, each with an hourly depth series, worst depth (p90), the hour it passes 15 cm, and a pre-wet flag.
18. **Features:**
19. a 162,067-node / 206,223-edge road graph with peak and hourly edge depths;
20. 168 dry-parking candidates (122 flyovers);
21. 46 major hospitals;
22. an 8,427-name place list.
23. **Navigation data** (`pipeline/05b_navigation.py`, aligned edge by edge with the graph):
24. one-way flags on 20,244 edges;
25. roundabouts: 248 edges;
26. tunnels/underpasses closed in rain: **280 edges**, including 47 OSM pieces named "Subway"/"Underpass" that were not tagged as tunnels;
27. slip roads: 1,038;
28. road numbers on 11,207 edges;
29. 546 points of interest (276 pharmacies, 199 fuel, 66 police, 5 fire).
30. **Cross-check:** ANUGA on a 103,159-triangle mesh over Velachery and Pallikaranai.
31. **Validation** on held-out wards (section 5).
32. **Export:**
33. depth PNG frames, terrain tiles, street JSON tiles (288×2 km), routing binaries;
34. `proof.json`, `runs.json`, `hospitals.json`, `parking.json`, `places.json`, `pois.json`;
35. the **zoom-14 basemap** (7.1 MB instead of 16.9 MB; used online too);
36. `offline/manifest.json`: 361 core files, 31.7 MB raw, 18.6 MB gzipped, versioned.

4.3 The live part (AWS)

- **forecast-check Lambda** (EventBridge Scheduler, every 3 h):
- reads Open-Meteo hourly rain for Velachery, T. Nagar, Anna Nagar and Tambaram;
- takes the 24 h mean and picks the two nearest modelled storms;
- writes `current.json` to S3;
- compares each subscriber's street risk band with the last run;
- Bedrock writes two sentences from the model's numbers, then SNS publishes with `segment_id`.
- **subscribe Lambda**: email + street → DynamoDB → an SNS email subscription with a filter policy on `segment_id`.
- **assistant Lambda**: Strands Agents on Bedrock with six model tools (`get_street_risk`, `find_dry_parking`, `safe_route`, `hospital_status`, `current_forecast`, `model_limits`). Numbers in its replies are checked against tool output.
- **geocode Lambda**: Amazon Location Places v2 (biased to Chennai), with a Nominatim fallback.

4.4 In the browser (and offline)

- **Map**:
- MapLibre GL JS 6 with a Protomaps basemap (zoom-14 PMTiles), raster-dem terrain, colour-relief, hillshade, 3D buildings.
- A three.js water layer blends hourly depth frames on the GPU.
- Canvas rain; PNG decoding in a worker; the map redraws only when something changes.
- **Service worker** (`web/src/sw.ts`; vite-plugin-pwa 1.3.0, Workbox 7.4.1, classic):
- precaches the app shell (50 entries, 3.7 MB: JS, MapLibre worker, CSS, fonts, map glyphs, icons);
- serves `hg-pack-<version>` first, then `hg-seen` (what you looked at; 900 entries, 30 days);
- fetches `current.json` network-first with a 4 s timeout;
- answers basemap byte ranges from the saved file.
- **Save for offline** (`offline/pack.ts`):
- up to 3 places; street tiles within 3 km + 600 m;
- every storm's street answers;
- the graph with all storms' depths, plus hourly depths for tonight's forecast and the three replays;
- terrain, water meta, parking, hospitals, places, POIs, the forecast.
- It resumes after interruption and is versioned (the old pack is kept until the new one is complete); it asks the browser not to evict it (`persist()`).
- **Navigation** (`lib/nav.ts` in a Web Worker):
- time-dependent search; each edge is judged at the hour you reach it, looking one hour ahead, using the worse storm of a blend;
- limits 10/10/20 cm; penalty $1 + 3(D/limit)^2$ from 5/5/10 cm; road-class weights prefer main roads;
- subways closed in rain; one-way rules for vehicles.
- Destinations: dry parking (dry in every storm of the blend), hospitals, high ground (a main-road junction whose roads stay under 5 cm all storm), saved places.
- The card's "move your car by" decision uses the same router (latest hour a car route exists).
- **Directions** (`lib/directions.ts`):
- legs by road name; GraphHopper turn thresholds;
- "take the 2nd left" counting real side streets, not driveways, capped at the third;
- roundabout exits; stretches under 15 m folded into the next instruction.
- **Live** (`ui/NavLive.tsx`): `watchPosition`, snapping to the route ahead, off-route replanning (50 m, or 30 m on foot, plus the GPS error, 3 fixes), `speechSynthesis`, screen wake lock, and a preview mode.

5. Numbers we may quote (with caveats)

5.1 Validation: the honest headline

The test: on GCC wards **never used for tuning** (the even-numbered wards), pick a street residents reported flooded in Dec 2015 and one they did not. How often does the model put more water on the reported one?

Map	Score	95% range (resampling whole wards)
HighGround model (2015 rain + release)	53%	51% to 55%
Coin toss / random map	50% (random map 49%)	
Low ground alone	47%	41% to 51%
Distance to the nearest canal or river	56% (better than the model)	53% to 60%

- **Verdict, word for word: "slightly better than chance."** Never quote 53% alone.
- **Sample:** 3,200 reported street segments in the held-out wards (27,970 unreported); 7,894 reported citywide.
- **The release matters by the river:** rain-driven streets score 54%. River-driven streets score **51% without the release, 53% with it.**
- **NRSC satellite extent:** the model covers 33% of it; a random map of the same size covers 31%; the lowest ground covers 21%.
- **GCC 2015 hotspots (327):** model 53%, low ground 55%, nearest channel 57%, random 49%.
- **Ward level:** Spearman 0.11, not significant.
- **Drains:** 10 mm/h is a stated assumption; every value from 0 to 30 mm/h scored the same.
- **Withdrawn metric:** our first score (33% vs 19%) rewarded patchy maps; a random speckle scored 50%. We withdrew it.

5.2 Second-model cross-check

ANUGA vs our solver (Velachery, 200 mm, first 14 h): they agree on **73%** of the cells where either model has ≥ 15 cm (critical success index). Depth correlation is 0.90. Same terrain, so this checks the arithmetic, not the terrain.

5.3 Storm outcomes

Storm	Rain	City land ≥ 15 cm	Hospitals cut off (of 46)
100 mm design	100 mm / 24 h	2.4%	0
150 mm design	150 mm / 24 h	9.5%	—
200 mm design	200 mm / 24 h	17%	9
300 mm design	300 mm / 24 h	25%	15
400 mm design	400 mm / 24 h	30%	16
Dec 2015, rain + release	374 mm	29%	16
Cyclone Michaung 2023	415 mm	23%	15
Cyclone Fengal 2024	114 mm	3.6%	0

Mean tide. Smaller storms are probably overstated, because drains are one uniform assumption.

5.4 Scale, performance and the new features (measured)

Scale:

- ~1.2 million cells; 371,679 buildings removed; 16 runs; mass error 0.000%;
- 168,156 street pieces; 206,223 road edges; 168 parking candidates; 46 hospitals.

Speed:

- ~57 fps on an integrated-GPU laptop with the water flowing;
- the answer card appears ~12 s after load with the opening flight.

Routing:

- median **28–33 ms per plan** in the browser; worst case ~3.4 s (a no-route case that also searches later hours, run in the worker, off the main thread);
- **36 random trips**, 3 storms × 3 modes × 3 destination kinds: **0** edges over the limit at the reached hour, **0** wrong-way one-ways, **0** subways;
- directions sweep of 48 trips: 0 back-to-back jogs, 0 subways.

Offline:

- app shell precache 3.7 MB;
- pack core 18.6 MB gzipped plus the saved street tiles;
- with only the pack on the phone, the offline gate passes **10/10**, 0 page errors: opens, card, battery-saver map, basemap, plan image, help, route with directions, search.
- Stored size after saving one place was **~83 MB** (measured before terrain and replay depths were added; a little more now; street files are stored uncompressed).

Demo streets (Michaung replay, verified in the search box):

Street	Card shows	Decision
Dhandeeswaram Nagar 8th Cross Road	118 cm	Velachery Upper Flyover (600 m) by Tue 1 AM
Velachery Main Road	116 cm	Phoenix Marketcity Parking (450 m) by Tue 1 AM
Arumugam Road	106 cm	Velachery MRTS Bridge (350 m) by Tue 1 AM

6. Tech stack

6.1 Used now

Layer	Tools (versions as installed)
Frontend	Vite 8.3.4 , React 19.3 , TypeScript 6 , MapLibre GL JS 6.13.0 , three.js 0.186.1 (custom water layer, GLSL), PMTiles 4.5.0 + Protomaps basemaps 5.7.2 , Web Workers (image decoding, navigation), Canvas 2D (rain, plan image)
Offline / PWA	vite-plugin-pwa 1.3.0 (injectManifest), Workbox 7.4.1 (precaching, routing, expiration, range requests), Cache API, StorageManager (persist, estimate), web app manifest + icons, Fontsource (@fontsource-variable/aneek-latin, @fontsource/hind 5.3.0), self-hosted Protomaps glyphs (Noto Sans) and sprites
Navigation / device APIs	Geolocation watchPosition, Screen Wake Lock, Web Speech speechSynthesis, Web Share (files and text), sms: / tel: / WhatsApp links, React portals

Layer	Tools (versions as installed)
Model pipeline	Python 3.11 , numba , NumPy, SciPy, pandas, rasterio/GDAL, GeoPandas, pyproj, scikit-image, pyosmium , pmtiles (Python, basemap cut); ANUGA 4.0.1 (separate env)
AWS (coded, not yet deployed)	S3 , CloudFront (OAC, CORS, gzipped routing binaries), Amplify Hosting (PWA headers: <code>sw.js/index.html</code> no-cache, assets immutable; rewrite rule fixed for <code>.mjs/.pbf/.webmanifest</code>), Lambda (Python 3.12 x4), API Gateway (HTTP API), DynamoDB , SNS (filter policy per street), EventBridge Scheduler (<code>rate(3 hours)</code>), Bedrock (Claude Sonnet: alerts + assistant), Strands Agents SDK , Amazon Location Service (Places v2), AWS SAM , AWS Open Data Registry (Copernicus DEM, ESA WorldCover), AWS Budgets (\$10 alarm)
Testing / tooling	Playwright 1.64 (hero test; offline gate; route rules; directions sweep; navigation UI, live, phone and demo-street checks; frame-perfect footage with a virtual clock), moto (mocked AWS), ffmpeg (OpenH264; rough cut), oxlint , Miniforge/conda, nvm/Node 24
Data / services	OpenStreetMap (Tamil Nadu extract), Open-Meteo forecast + ERA5, IMD totals, OpenCity datasets (2015 crowd reports, GCC hazard zones, NRSC 2015 extent, GCC wards, hotspots, stagnation points)
AI during the build	Claude Code with reviewer subagents (model-reviewer, design-critic, qa-tester, judge) and research agents (creative ideas, offline web tech, flood navigation, Chennai ground truth)

6.2 Planned (will be used), in priority order

What	Tool / service	Why
Official warnings in the app and in every offline pack	Lambda ingests NDMA SACHET CAP RSS (IMD Chennai alerts; the feed has no CORS, so a server is required) into <code>current.json</code>	Point to official channels, keep them offline
Push "your street's risk changed" to phones	Web Push from the forecast Lambda (pywebpush 2.5.0, VAPID key in SSM Parameter Store , subscriptions in DynamoDB); Periodic Background Sync on Android	Updates reach offline packs before the towers go
Relief centres as destinations	GCC relief-centre list (OpenCity PDF, Oct 2024) geocoded with Amazon Location , spot-checked, labelled "call 1913 to check it is open"	The most-asked destination in floods
Typo-tolerant offline search	uFuzzy 1.0.19 (3.9 KB) with name normalisation (Velacheri/Velachery...)	Search works offline for misspellings
Smaller stored packs	<code>CompressionStream</code> in the pack downloader	~83 MB → ~15–20 MB stored
Tamil (and Hindi)	i18n of all copy; Noto Sans Tamil glyphs	About 18.5% of Tamil Nadu speaks English
SMS alerts	AWS End User Messaging SMS after TRAI DLT registration (needs a registered entity)	Feature phones, data outages
Voice line	Amazon Connect IVR (Indian numbers take weeks)	No smartphone needed
Screen-off navigation	Native wrapper (Capacitor)	Web pages pause GPS when the screen is off
Better hazard	Depth × velocity per edge from the solver (it already computes flux); citywide "move your car by" layer precomputed in Lambda to S3	Safer walking limits; the map as a decision layer
Better terrain	Revisit the v4 changes (grey opening, bank-level channels) with a new pre-registered test; finer DEM if one becomes open	Fix the Velachery/T. Nagar failure
Crowd reports (after moderation design)	Reports that can only make routes more cautious, photo required, 3 h expiry	Ground truth during storms

7. Honesty: limits, claims to make, claims to avoid

Limits (must appear in the app, video or page):

- 30 m satellite terrain: kerbs, culverts, small dips and **underpasses are invisible** (subways are closed in rain by name instead).
- Drains are one uniform 10 mm/h assumption; smaller storms are probably overstated.
- The model does not reproduce GCC's finding that Velachery floods far more than T. Nagar. Some flooded places read dry; don't demo Pallikaranai or Kotturpuram.
- The 2015 release is modelled from the CAG timeline. There is no storm surge, and rain is uniform across the city.
- Routes avoid streets **the model expects** to flood. They cannot see fallen trees, live wires, open drains or closures.
- Offline answers are as of the last saved forecast; nothing is live offline.
- English only.
- **Not an official warning. Follow GCC, IMD and TNSDMA advisories. In danger, call 112.**

Claims to make:

- street-by-street answers and decisions in plain words;
- tested honestly on held-out wards, and we show the modest result;
- an independent second model agrees on 73% of the flooded area;
- routes that avoid modelled water, by mode, offline;
- AWS-native; open data, open-source tools;
- email alerts (SMS coming; needs DLT).

Claims to avoid:

- "accurate", "predicts exactly", "official", "better than CFLOWS", "the only street-level forecast";
- "real-time flood model" (the model is precomputed; the forecast check is live);
- "AI predicts floods" (the AI only explains the model);
- "calibrated";
- **"safe route" or "dry route"** (say "a route that avoids streets our model expects to flood");
- "flyover parking is allowed";
- "works on every phone offline" (iPhone must add it to the home screen);
- "live updates during an outage";
- SMS or Cell Broadcast as working;
- any validation number without its caveat.

8. What fought back (for the blog and the video)

1. **Satellite elevation is full of fake lakes.** Closed hollows could hold 124 million m³, about 70% of a 200 mm storm. We capped them, carved drains by type and opened the edges.
2. **Channels leaked before the rain.** About 2% of the city was flooded at 6 PM in every storm. Fixed, and everything rerun.
3. **Our first validation metric was wrong.** It rewarded patchy maps (random noise beat the model). A reviewer caught it; we switched to a ranking test and stopped calling the drains "calibrated".
4. **A 2-core laptop.** 35 → ~11 min per storm. A review then found sloshing, so CFL is 0.5 (~15 min).
5. **The model's edges leaked** (a third of the 2015 release; the sea through Ennore). Fixed.

6. **The sea was drawn as floodwater** in high-tide frames. Masked.
 7. **A better-looking terrain (v4) we did not ship.** The rule was written before the run; v4 missed one condition, so v3 stayed.
 8. **Lag** came from rain forcing full-map redraws and PNG decoding on the main thread. Fixed with a separate canvas, render-on-demand and a decode worker.
 9. **"Nothing animates"** on a laptop with desktop animations off. The app now offers "Play animations".
 10. **Offline found two deploy-breaking bugs before any deploy:**
 - Amplify's rewrite rule would have served MapLibre's `.mjs` worker as the web page, so the map would never have started.
 - With only the saved pack, the map never finished loading because terrain tiles were missing.
 11. **OpenStreetMap splits a street into pieces.** The top demo street read 118 cm on one piece and 0 cm on the next. Search now answers for the street's deepest stretch and says so.
 12. **47 of 69 subway pieces weren't tagged as tunnels.** Routes went through Madley and Aranganathan subways until they were closed by name.
 13. **The card and the route disagreed** ("by 2 AM" vs "under water at 2 AM") because they used different rules. The card's decision now comes from the navigation router.
-

9. Reviews and QA (process)

After each piece of work a reviewer ran, and every blocker and major was fixed. Full logs are in PROGRESS.md.

model-reviewer (×2):

- withdrew the gameable metric; fixed the release leak, sloshing, the sea short circuit and the pre-storm mask;
- made "leave by" use model-grid depths;
- insisted on honest wording and on disclosing the Velachery/T. Nagar failure.

design-critic (×3). Latest round on offline and navigation, 2 blockers and 5 majors:

- a practice-plan band and date on shared plans;
- wire and drain warnings in the live banner;
- a shorter card with "Take me there";
- card and route agreeing;
- no "safe/dry route" wording;
- a glanceable banner;
- the Go panel replacing the card body.

qa-tester (×5). Latest round on offline and navigation, 1 blocker and 9 majors, all fixed:

- the pack now includes terrain and replay depths, so it opens with nothing else cached;
- honest messages for streets that weren't saved;
- failed fetches are retried;
- subways closed by name;
- cleaner directions;
- a banner when there is no route;
- battery-saver notes;
- the card's depth used in navigation.

judge (x2). Top suggestions acted on: time-aware routing (#1); search fix, verified demo streets, footage and rough cut (#2).

Test scripts (web/tests/):

Script	What it checks
hero.mjs	The hero flow (9/9)
offline.mjs	The offline gate, pack-only, 10/10
route_rules.mjs	Routing rules
directions_sweep.mjs	Directions quality across many trips
nav_ui.mjs, nav_live.mjs	Navigation UI and live directions
mobile_new.mjs	Phone layout
demo_streets.mjs	Demo streets via the real search box
reduced_motion.mjs	Devices that ask for less motion
record_frames.mjs	The footage recorder

The Lambda flow is tested against moto in infra/tests/test_lambdas_moto.py.

10. Repository map

CLAUDE.md	original spec · PROGRESS.md engineering log · README.md public overview
pipeline/	
00_fetch.py ... 07_export_tiles.py	data → terrain → model → streets → graph/parking/hospitals → validation → tiles
05b_navigation.py	one-way/roundabout/tunnel(+named subway)/slip flags, road numbers, POIs (aligned with the graph)
08_offline_assets.py	zoom-14 basemap cut + offline/manifest.json (versioned pack list)
gates.py, config.yaml, run_all.sh, post_runs.sh, v4_*.py/sh, common.py, scoring.py	
infra/	
template.yaml	SAM: S3, CloudFront, DynamoDB, SNS, HTTP API, 4 Lambdas, Scheduler, Amplify (+ PWA headers)
deploy.sh	one command: budget, Bedrock model pick, sam deploy, upload (gzipped graph), Amplify
functions/{subscribe,forecast_check,assistant,geocode}/app.py · layers/common/python/hg.py · tests/	
web/	
src/App.tsx, main.tsx	screens, opening, online status, battery saver, service worker registration
src/sw.ts	service worker (precache, packs, forecast, basemap ranges)
src/offline/	pack.ts (save/update/remove), status.ts (online probe, forecast age), register.ts
src/map/	MapView (camera, layers, battery saver, route framing), style (night palette, self-hosted glyphs)
src/water/	three.js water layer, shaders, rain
src/screens/	Tonight, WhatIf, Proof, Hospitals, About
src/ui/	GoPanel, NavLive, StepIcon, HelpCard, PlanCard, OfflineSave, Search, Timeline, Timelapse, DepthGlyph, Legend, Readout, Subscribe, Assistant
src/lib/	nav.ts (router), nav.worker.ts, navClient.ts, directions.ts, routing.ts, streets.ts, data.ts, scenario.ts, words.ts, geo.ts
public/basemap-assets/	Noto Sans glyphs + sprites · public/icons/ app icons
tests/	Playwright scripts (section 9)
docs/	this audit (+PDF), offline/navigation plan (+PDF), landing context (+PDF), VIDEO_CUT.md, DEMO.md, blog draft, architecture.png, landing-assets/, video/ (720p rough cut), make_*.py (PDFs, demo guide, rough cut, architecture)
review/	screenshots and reports per phase and review; p5/footage/*.mp4 (full shots, not in git)
data/	raw data and model outputs (not in git; served from data/out/web)

11. How to run

```
# web (data already built)
cd web && npm install
npx vite --port 5173
npx vite build && npx vite preview --port 4173
http://127.0.0.1:4173/?replay=michaung2023

# dev (no service worker)
# production build: service worker, offline, recording
# Michaung replay (add &motion=on if the device reduces motion)

# tests
HG_QUERY='?replay=michaung2023' node tests/hero.mjs http://127.0.0.1:4173/
node tests/offline.mjs http://127.0.0.1:4173/ # offline gate (pack only)
node tests/route_rules.mjs http://127.0.0.1:5173/ # routing rules (dev server)
node tests/directions_sweep.mjs http://127.0.0.1:5173/
source ../pipeline/env.sh && python ../infra/tests/test_lambdas_moto.py
```

```
# offline / navigation assets after a model re-export
python pipeline/05b_navigation.py && python pipeline/08_offline_assets.py

# footage and rough cut
WARM=1 node tests/record_frames.mjs <opening|search|navigate|timelapse|whatif|proof> http://127.0.0.1:4173/ ../review/p5/footage
python docs/make_rough_cut.py # → review/p5/footage/rough_cut.mp4 + docs/VIDEO_CUT.md

# AWS (after section 16)
infra/deploy.sh
```

Gotchas:

- After swapping `data/out/web`, restart Vite.
- Run long jobs under `systemd-inhibit --what=sleep`.
- Never `pgkill -f` a pattern that appears in your own command line.

12. Pitch (deck, landing page or a 60-second talk)

1. **Hook:** "Every monsoon, Chennai parks its cars on flyovers, because nobody tells residents where the water will go."
2. **Problem:** official systems warn that a flood is coming. Residents still guess what it means for their street, their car, their way out and their hospital. And when the power and towers go, as they did in 2015 and 2023, they're on their own.
3. **Solution:** type your street and watch tonight's storm rise on a 3D model of the city. You get a plain answer and a decision: how deep, when it's too deep for scooters, "move your car to X by 2 AM".
Take me there gives a route around the water, on foot, by two-wheeler or by car, with directions. Plus the hospitals you can still reach, and an email if it changes.
4. **When the network dies:** save your area before the storm. HighGround opens from the phone, still plans a way out, gives the help numbers, and makes a flood plan image for the family group.
5. **How:** a real 2D flood model of all of Greater Chennai, built on open data from AWS, with 16 storms precomputed. AWS checks the live forecast every 3 hours, and an assistant can only quote the model.
6. **Proof, honestly:** tested on wards we never tuned on against 2015 reports, it is slightly better than chance at street level (53% vs 50%), and we show that on screen. A second, independent model agrees on 73% of the flooded area.
7. **Built on AWS:** S3, CloudFront, Amplify, Lambda, API Gateway, DynamoDB, SNS, EventBridge Scheduler, Bedrock, Strands Agents, Amazon Location, SAM, AWS Open Data.
8. **Next:** official IMD warnings in the app via SACHET, push alerts, relief centres, Tamil, SMS once DLT-registered, better terrain.

13. The 3-minute video

Rough cut: `review/p5/footage/rough_cut.mp4` (1080p) and `docs/video/HighGround_rough_cut_720p.mp4`, 2:30, no audio. The cut list with voice-over lines is in `docs/VIDEO_CUT.md`.

Starts	Shot	Voice-over
0:00	Title cards: flyovers (replace the first with a photo of cars on the Velachery flyover, credited)	"Every monsoon, Chennai parks its cars on flyovers... because nobody tells residents where the water will go."

Starts	Shot	Voice-over
0:11	<code>opening.mp4</code> : Bay of Bengal → Velachery, the night plays, the card	Michaung replayed as if tonight; model runs from S3/CloudFront
0:29	<code>search.mp4</code> : Arumugam Road → 106 cm, "Move your car to Velachery MRTS Bridge (350 m) by Tue 1 AM"	Amazon Location search; the decision
0:43	<code>navigate.mp4</code> : Take me to dry ground → Preview the drive with directions	"A route that avoids streets the model expects to flood. It works offline."
1:01	SLOT: the real alert email (#admin → Run and send alerts, 300 mm)	EventBridge → Lambda → Bedrock → SNS
1:11	Title card: 712/1,814 feeders, 30% of 42,747 towers; offline stills (battery-saver map, offline route, flood plan)	"When the power goes, the towers go. Saved before the storm, HighGround keeps working."
1:27	<code>whatif.mp4</code> + Hospitals still	"What if 400 mm fell tonight? At 300 mm, 15 of 46 major hospitals can't be reached by car."
1:39	<code>timelapse.mp4</code> : Dec 2015 with the release	The 2015 floods hour by hour
1:53	<code>proof.mp4</code> : the swipe	"53% against a coin toss's 50%: slightly better than chance, and we show it."
2:03	SLOT: Ask HighGround answering with grounded numbers	Strands Agents on Bedrock
2:13	<code>docs/architecture.png</code>	Name every AWS service
2:23	Close card	"Not an official warning. In danger, call 112."

The remaining ~30 s is for voice-over pacing or a real-phone offline beat (airplane mode on an Android phone, after deploy). Re-record any shot with `record_frames.mjs` and rebuild with `docs/make_rough_cut.py`.

14. Open issues and to-do (owner)

#	Item	Owner
1	Create the AWS profile <code>highground</code> (us-east-1), put a real <code>ALERT_TEST_EMAIL</code> in <code>.env</code> , enable Bedrock Claude Sonnet	User, now
2	<code>infra/deploy.sh</code> → live URL; subscribe once and click the SNS confirmation link; film the email and the assistant into the two slots	Claude (deploy) + user (inbox, filming)
3	Post-deploy check of the live site: hero, offline on a real Android phone, assistant, email	Claude + user
4	Edit the video (voice-over, the two slots, optional flyover photo); submit by 20:00	Teammate 1
5	Blog: add the offline/navigation paragraph and the "what fought back" items 10–13; publish on AWS Builder Center; link it	Teammate 1
6	Landing page with the positioning update (context PDF sent)	Teammate
7	Live URL and video link into README, <code>LANDING_PAGE_CONTEXT</code> , this audit	Claude, after deploy
8	Architecture diagram: add the service worker + offline pack box	Claude, optional
9	Known minors: a U-turn on a flyover deck when the parking target is the deck; the speckled city view at low zoom (the model's pockets); answer at ~8 s with motion off; English only	Later
10	Student verification on AWS Builder Center	User

15. Timeline (what was built when)

When	What
Fri 9 Oct	Data, terrain, solver, 16 storms, calibration withdrawn → assumption, ANUGA cross-check, validation, SAM stack, first web app, first reviews
Sat 10 Oct	v3 rebuild, v4 test (rejected by rule), UX overhaul (camera travel, plain words, palette, transitions), time-lapse, rain and water fixes, motion offer, landing context
Sun 11 Oct, 00:00–03:05	Offline (Slice 1), navigation (Slice 2), live directions (Slice 3)
Sun 11 Oct, 07:40–08:45	Design and QA review fixes
Sun 11 Oct, 08:30–09:40	Judge #2, search fix, verified demo streets, footage, rough cut

16. FOR THE USER (only you can do these)

1. `aws configure --profile highground`, with access keys for the team's AWS account and region `us-east-1`. (As of 09:45 the profile does not exist.)
2. Put your real inbox in `.env: ALERT_TEST_EMAIL=you@yourdomain` (still the placeholder).
3. In the AWS console (us-east-1): Amazon Bedrock → Model access → enable an Anthropic Claude Sonnet model.
4. Tell Claude. It runs `infra/deploy.sh`, which:
5. creates the \$10 budget alarm;
6. deploys the stack;
7. uploads the model outputs, including the offline pack files and the gzipped graph;
8. builds and deploys the PWA to Amplify;
9. runs the first forecast check;
10. prints the live URL. The admin token is in `data/admin_token`.
11. Subscribe once on the live site with that email and **click the AWS Notifications confirmation link**. Then `#admin` → "Run and send alerts" (300 mm) and film the email.
12. Verify student status on AWS Builder Center.

17. Attribution lines (required wherever data or the map appears)

Source	Credit line
Elevation	Copernicus DEM GLO-30, © DLR e.V. 2010–2014 and © Airbus Defence and Space GmbH 2014–2018, provided under COPERNICUS by the European Union and ESA; all rights reserved. Read from the AWS Open Data registry
Land cover	ESA WorldCover 2021 (CC BY 4.0), © ESA WorldCover project / contains modified Copernicus Sentinel data (2021), from the AWS Open Data registry
Map data	© OpenStreetMap contributors (ODbL); basemap by Protomaps; Noto Sans fonts (OFL)
Rain	Open-Meteo (CC BY 4.0), including ERA5 (Copernicus Climate Change Service)
Validation data	2015 flood reports: OpenCity, osm-in/flood-map contributors. Hazard zones, wards, hotspots: Greater Chennai Corporation via OpenCity. 2015 inundation extent: NRSC via OpenCity

Source	Credit line
Cross-check model	ANUGA, Geoscience Australia
News facts used on title cards	The News Minute, The Week/PTI, Deccan Herald (links in section 1)

18. Glossary (for writing copy)

- **CFLOWS:** Chennai's flood decision tool for officials (NCCR). **Chennai Flood Monitor:** the state's public forecasting portal. **TN-Alert:** the state's alert app.
- **Too deep for scooters:** 15 cm. **Cars stall:** about 30 cm (the display bands). **Route limits:** 10 cm on foot or two-wheeler, 20 cm by car (stricter on purpose).
- **Replay:** a real past storm shown as if it started at 6 PM tonight. **Practice plan:** a flood plan made from a replay.
- **Move your car by:** the latest hour a car route to the dry place still avoids water over the car limit.
- **Save for offline / pack:** files kept on the phone for chosen places. **Battery saver:** flat map, streets coloured by depth.
- **Held-out wards:** even-numbered GCC wards, never used for tuning. **CSI:** of the area either model floods, the share both flood.
- **Cusecs:** cubic feet per second. **Pre-wet street:** low ground the model holds ≥ 5 cm on before the rain.